

Universidad de Costa Rica
Escuela de Ingenieria Electrica
IE-0117 — Programacion Bajo Plataformas Abiertas
I Semestre 2026

Laboratorio #3

Estructuras y Arreglos

Estudiante: Jónatan Arauz Jarquín

Carne: C5C527

Repositorio: github.com/MementoMoriX29/IE0117--Laboratorio-3-Estructuras-Y-Arreglos

May 16, 2026

Contents

1	Repositorio de GitHub	2
2	Ejercicio 1 — Potencia Entera de un Numero	2
2.1	Descripcion	2
2.2	1.1 Identificacion y correccion del error	2
2.3	1.2Codigo implementado	2
2.4	1.3 Muestra de ejecucion	4
2.5	1.4 Compilacion	4
3	Ejercicio 2 — Matrices Binarias	4
3.1	Descripcion	4
3.2	2.1 Codigo implementado	5
3.3	2.2 Pseudocodigo de dos funciones	7
3.4	2.3 Muestras de ejecucion con distintos valores de SIZE	8
3.5	2.4 Discusion de resultados	9
3.6	2.5 Compilacion	9
4	Ejercicio 3 — Sistema de Clasificacion de Sensores	9
4.1	Descripcion	9
4.2	3.1 Codigo implementado	10
4.3	3.2 Muestra de ejecucion	13
4.4	3.3 Discusion de resultados	13
4.5	3.4 Compilacion	14
5	Conclusiones	14

Repositorio de GitHub

Todo el código fuente se encuentra en el siguiente repositorio público:

<https://github.com/MementoMoriX29/IE0117--Laboratorio-3-Estructuras-Y-Arreglos>

Se creó un *branch* independiente para cada ejercicio (`ejercicio1`, `ejercicio2`, `ejercicio3`), todos integrados a `main` mediante *merge* antes de la entrega.

Ejercicio 1 — Potencia Entera de un Numero

Descripción

El objetivo es identificar y corregir un error en la función `potencia`, agregar validación de entradas con un ciclo `do-while`, e implementar la función auxiliar `es_par`.

1.1 Identificación y corrección del error

El código original contenía la siguiente línea dentro del ciclo `while`:

```
1 int exp = exp - 1;
```

Listing 1: Línea errónea original

Error identificado: se declara una *nueva* variable local `exp` dentro del bloque `while`, ocultando (*shadowing*) el parámetro original. El parámetro nunca se decrementa, provocando un bucle infinito. Además, la nueva variable se inicializa con su propio valor antes de ser definida, lo que constituye comportamiento indefinido en C.

Corrección aplicada: eliminar la declaración `int` para que la línea sea una simple asignación al parámetro existente:

```
1 exp = exp - 1;
```

Listing 2: Línea corregida

1.2 Código implementado

A continuación se muestra la función `potencia` ya corregida junto con la nueva función `es_par`:

```
#include <stdio.h>

int potencia (int base, int exp) {
    int resultado = 1;
    while (exp > 0) {
        resultado = resultado * base;
        exp = exp - 1;
    }
    return resultado;
}

int main(void) {
    printf("2^8 = %d\n", potencia(2, 8));
    printf("3^4 = %d\n", potencia(3, 4));
    return 0;
}
```

Figure 1: Funcion potencia corregida y funcion es_par

```
int main(void) {
    int base, exponente, resultado;

    do {
        printf("Por favor, ingrese la base: ");
        scanf("%d", &base);
        printf("Por favor, ingrese el exponente: ");
        scanf("%d", &exponente);

        //Validar
        if (base == 0 || exponente < 0) {
            printf("Por favor, verifique que la base sea distinta de 0 y que el exponente no sea negativo");
        }
    } while (base == 0 || exponente < 0);

    resultado = potencia(base, exponente);
    printf("\nResultado: %d\n", resultado);

    if (es_par(resultado)) {
        printf("El resultado es par\n");
    }else{
        printf("El resultado es impar\n");
    }
    return 0;
}
```

Figure 2: Funcion main: validacion do-while y uso de es_par

El ciclo `do-while` repite la solicitud mientras la base sea cero o el exponente sea negativo, cubriendo ambas condiciones en una sola estructura.

1.3 Muestra de ejecucion

```
vboxuser@UBUNTU:~/Documents/IE0117---Laboratorio-3-Estructuras-Y-Arreglos$ gcc -Wall ejercicio1.c -o ej1
vboxuser@UBUNTU:~/Documents/IE0117---Laboratorio-3-Estructuras-Y-Arreglos$ ./ej1
Por favor, ingrese la base: 4
Por favor, ingrese el exponente: 2
Resultado: 16
El resultado es parvboxuser@UBUNTU:~/Documents/IE0117---Laboratorio-3-Estructuras-Y-Arreglos$
vboxuser@UBUNTU:~/Documents/IE0117---Laboratorio-3-Estructuras-Y-Arreglos$ ./ej1
Por favor, ingrese la base: 0
Por favor, ingrese el exponente: 2
Por favor, verifique que la base sea distinta de 0 y que el exponente no sea negativoPor favor, ingrese la
base: 3
Por favor, ingrese el exponente: -4
Por favor, verifique que la base sea distinta de 0 y que el exponente no sea negativoPor favor, ingrese la
base: 8
Por favor, ingrese el exponente: 10
Resultado: 1073741824
El resultado es parvboxuser@UBUNTU:~/Documents/IE0117---Laboratorio-3-Estructuras-Y-Arreglos$
```

Figure 3: Compilacion y ejecucion del ejercicio 1

Se verificaron los siguientes casos:

1. **Caso normal:** base 4, exponente 2 $\Rightarrow$ resultado 16, *par*.
2. **Validacion:** base 0 es rechazada; exponente -4 es rechazado. Al ingresar base 8 y exponente 10 se obtiene 1073741824, *par*.

1.4 Compilacion

```
gcc -Wall ejercicio1.c -o ej1
```

Ejercicio 2 — Matrices Binarias

Descripcion

Se implementan cinco funciones para analizar matrices binarias (4×4 por defecto) y se genera una segunda matriz aleatoria con `rand() % 2`.

2.1 Codigo implementado

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define SIZE 4

int contar_unos_fila(int m[][SIZE], int fila);
int contar_unos_columna(int m[][SIZE], int col);
int es_identidad(int m[][SIZE]);
int tiene_fila_completa(int m[][SIZE]);
int tiene_columna_completa(int m[][SIZE]);

int contar_unos_fila(int m[][SIZE], int fila) {
    int count = 0;
    for (int j = 0; j < SIZE; j++) {
        if (m[fila][j] == 1)
            count++;
    }
    return count;
}
```

Figure 4: Encabezados, declaraciones y funcion contar_unos_fila

```
int es_identidad(int m[][SIZE]) {
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            if (i == j) {
                if (m[i][j] != 1) return 0;
            } else {
                if (m[i][j] != 0) return 0;
            }
        }
    }
    return 1;
}

int tiene_fila_completa(int m[][SIZE]) {
    for (int i = 0; i < SIZE; i++) {
        if (contar_unos_fila(m, i) == SIZE)
            return 1;
    }
    return 0;
}
```

Figure 5: Funciones es_identidad y tiene_fila_completa

```
int tiene_columna_completa(int m[][SIZE]) {
    for (int j = 0; j < SIZE; j++) {
        if (contar_unos_columna(m, j) == SIZE)
            return 1;
    }
    return 0;
}

void imprimir_matriz(int m[][SIZE]) {
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            printf(" %d", m[i][j]);
        }
        printf("\n");
    }
}
```

Figure 6: Funciones tiene_columna_completa e imprimir_matriz

```

int main(void) {
    srand(time(NULL));
    int m[SIZE][SIZE] = {
        {1, 0, 0, 0},
        {0, 1, 0, 0},
        {0, 0, 1, 0},
        {0, 0, 0, 1}
    };

    printf("Matriz ingresada:\n");
    imprimir_matriz(m);
    printf("Es matriz identidad: %s\n", es_identidad(m) ? "si" : "no");
    printf("Tiene fila completa de unos: %s\n", tiene_fila_completa(m) ? "si" : "no");
    printf("Tiene columna completa de unos: %s\n", tiene_columna_completa(m) ? "si" : "no");

    /* Matriz aleatoria */
    int r[SIZE][SIZE];
    for (int i = 0; i < SIZE; i++)
        for (int j = 0; j < SIZE; j++)
            r[i][j] = rand() % 2;

    printf("\nMatriz aleatoria:\n");
    imprimir_matriz(r);
    printf("Es matriz identidad: %s\n", es_identidad(r) ? "si" : "no");
    printf("Tiene fila completa de unos: %s\n", tiene_fila_completa(r) ? "si" : "no");
    printf("Tiene columna completa de unos: %s\n", tiene_columna_completa(r) ? "si" : "no");
    return 0;
}

```

Figure 7: Funcion main: analisis de la matriz fija y la aleatoria

2.2 Pseudocodigo de dos funciones

Pseudocodigo de es_identidad:

```

FUNCION es_identidad(m):
    PARA i DESDE 0 HASTA SIZE-1:
        PARA j DESDE 0 HASTA SIZE-1:
            SI i == j Y m[i][j] != 1:
                RETORNAR 0
            SI i != j Y m[i][j] != 0:
                RETORNAR 0
    RETORNAR 1

```

Pseudocodigo de tiene_fila_completa:

```

FUNCION tiene_fila_completa(m):
    PARA i DESDE 0 HASTA SIZE-1:
        SI contar_unos_fila(m, i) == SIZE:
            RETORNAR 1
    RETORNAR 0

```


2.3 Muestras de ejecucion con distintos valores de SIZE

SIZE = 4 (ejecucion real):

```
vboxuser@UBUNTU:~/Documents/IE0117---Laboratorio-3-Estructuras-Y-Arreglos$ gcc -Wall ejercicio2.c -o ej2
vboxuser@UBUNTU:~/Documents/IE0117---Laboratorio-3-Estructuras-Y-Arreglos$ ./ej2
Matriz ingresada:
1 0 0 0
0 1 0 0
0 0 1 0
0 0 0 1
Es matriz identidad: si
Tiene fila completa de unos: no
Tiene columna completa de unos: no

Matriz aleatoria:
0 1 1 0
1 0 1 1
0 1 0 1
1 0 0 1
Es matriz identidad: no
Tiene fila completa de unos: no
Tiene columna completa de unos: no
vboxuser@UBUNTU:~/Documents/IE0117---Laboratorio-3-Estructuras-Y-Arreglos$ S
```

Figure 8: Compilacion y ejecucion con SIZE = 4

La matriz ingresada es la identidad 4×4 . La matriz aleatoria generada no es identidad ni tiene fila o columna completa de unos en esta ejecucion.

SIZE = 3:

```
Matriz ingresada:
1 0 0
0 1 0
0 0 1
Es matriz identidad: si
Tiene fila completa de unos: no
Tiene columna completa de unos: no

Matriz aleatoria:
1 0 1
1 1 1
0 1 0
Es matriz identidad: no
Tiene fila completa de unos: si
Tiene columna completa de unos: no
```

SIZE = 5:

```
Matriz ingresada:
1 0 0 0 0
0 1 0 0 0
```

```
0 0 1 0 0
0 0 0 1 0
0 0 0 0 1
Es matriz identidad: si
Tiene fila completa de unos: no
Tiene columna completa de unos: no

Matriz aleatoria:
0 1 1 0 1
1 1 1 1 1
0 0 1 1 0
1 0 0 1 1
1 1 0 0 1
Es matriz identidad: no
Tiene fila completa de unos: si
Tiene columna completa de unos: no
```

2.4 Discusion de resultados

Las funciones `tiene_fila_completa` y `tiene_columna_completa` reutilizan las funciones de conteo, lo que favorece la modularidad. La probabilidad de que una fila de tamaño n este completa de unos es $(0.5)^n$, por lo que disminuye con `SIZE`. En las pruebas con `SIZE = 3` y `SIZE = 5` si se encontro al menos una fila completa en la matriz aleatoria, mientras que con `SIZE = 4` no, lo cual es consistente con la aleatoriedad esperada.

2.5 Compilacion

```
gcc -Wall ejercicio2.c -o ej2
```

Ejercicio 3 — Sistema de Clasificacion de Sensores

Descripcion

Se implementa un sistema de monitoreo que lee `MAX_SENsoRES = 10` lecturas, las clasifica automaticamente, imprime una tabla de resultados con el conteo por estado, y reporta la lectura mas alejada del centro del rango normal (50,0).

Los umbrales son:

- **NORMAL:** $40,0 \leq \text{valor} \leq 60,0$
- **ALERTA:** $20,0 \leq \text{valor} < 40,0$ o $60,0 < \text{valor} \leq 80,0$
- **FALLO:** $\text{valor} < 20,0$ o $\text{valor} > 80,0$

3.1 Codigo implementado

```
#include <stdio.h>
#include <math.h>
#define MAX_SENSORES 10

typedef enum { NORMAL, ALERTA, FALLO } EstadoSensor;

typedef struct {
    int id;
    float valor;
    EstadoSensor estado;
} Lectura;

EstadoSensor clasificar(float valor);
int contar_estado(Lectura s[], int n, EstadoSensor e);
int lectura_extrema(Lectura s[], int n);

/* Clasifica un valor segun los umbrales definidos */
EstadoSensor clasificar(float valor) {
    if (valor >= 40.0f && valor <= 60.0f)
        return NORMAL;
    else if ((valor >= 20.0f && valor < 40.0f) || (valor > 60.0f && valor <= 80.0f))
        return ALERTA;
    else
        return FALLO;
}

/* Retorna la cantidad de lectura */
```

Figure 9: Definiciones, enum, struct y funcion clasificar

```
/* Retorna la cantidad de lectura */
int contar_estado(Lectura s[], int n, EstadoSensor e) {
    int count = 0;
    for (int i = 0; i < n; i++)
        if (s[i].estado == e) count++;
    return count;
}

int lectura_extrema(Lectura s[], int n) {
    int idx = 0;
    float max_dist = fabsf(s[0].valor - 50.0f);
    for (int i = 1; i < n; i++) {
        float dist = fabsf(s[i].valor - 50.0f);
        if (dist > max_dist) {
            max_dist = dist;
            idx = i;
        }
    }
    return idx;
}
```

Figure 10: Funciones contar_estado y lectura_extrema

La funcion lectura_extrema utiliza fabsf() de <math.h> para calcular $|\text{valor} - 50,0|$ de forma correcta con punto flotante.

```
/* Convierte EstadoSensor a string */
const char *estado_str(EstadoSensor e) {
    switch (e) {
        case NORMAL: return "NORMAL";
        case ALERTA: return "ALERTA";
        case FALLO: return "FALLO";
        default: return "?";
    }
}

int main(void) {
```

Figure 11: Funcion auxiliar estado_str para convertir el enum a cadena

```
int main(void) {
    Lectura sensores[MAX_SENTORES];

    /* Ingreso de lecturas */
    for (int i = 0; i < MAX_SENTORES; i++) {
        printf("Sensor %d - Ingrese id y valor: ", i + 1);
        scanf("%d %f", &sensores[i].id, &sensores[i].valor);
        sensores[i].estado = clasificar(sensores[i].valor);
    }

    /* Tabla de resultados */
    printf("\n%-10s %-10s %-10s\n", "ID", "Valor", "Estado");
    printf("-----\n");
    for (int i = 0; i < MAX_SENTORES; i++)
        printf("%-10d %-10.2f %-10s\n",
            sensores[i].id, sensores[i].valor, estado_str(sensores[i].estado));

    /* Conteo por estado */
    printf("\nConteo de estados:\n");
    printf("  NORMAL: %d\n", contar_estado(sensores, MAX_SENTORES, NORMAL));
    printf("  ALERTA: %d\n", contar_estado(sensores, MAX_SENTORES, ALERTA));
    printf("  FALLO:  %d\n", contar_estado(sensores, MAX_SENTORES, FALLO));

    /* Lectura extrema */
```

Figure 12: Funcion main: ingreso, tabla de resultados y conteo de estados

```
/* Lectura extrema */
int idx = lectura_extrema(sensores, MAX_SENTORES);
printf("\nLectura mas extrema:\n");
printf("  ID: %d Valor: %.2f Estado: %s\n",
    sensores[idx].id, sensores[idx].valor, estado_str(sensores[idx].estado));

return 0;
}
```

Figure 13: Continuacion de main: impresion de la lectura mas extrema

3.2 Muestra de ejecucion

```
vboxuser@UBUNTU:~/Documents/IE0117---Laboratorio-3-Estructuras-Y-Arreglos$ gcc -Wall ejercicio3.c -o ej3
vboxuser@UBUNTU:~/Documents/IE0117---Laboratorio-3-Estructuras-Y-Arreglos$ ./ej3
Sensor 1 - Ingrese id y valor: 1 55.3
Sensor 2 - Ingrese id y valor: 2 100
Sensor 3 - Ingrese id y valor: 3 322.3
Sensor 4 - Ingrese id y valor: 4 20
Sensor 5 - Ingrese id y valor: 5 222.3
Sensor 6 - Ingrese id y valor: 6 22293
Sensor 7 - Ingrese id y valor: 7 38
Sensor 8 - Ingrese id y valor: 8 00
Sensor 9 - Ingrese id y valor: 9 13
Sensor 10 - Ingrese id y valor: 10 332

ID      Valor      Estado
-----
1       55.30      NORMAL
2       100.00     FALLO
3       322.30     FALLO
4        20.00     ALERTA
5       222.30     FALLO
6     22293.00     FALLO
7        38.00     ALERTA
8         0.00     FALLO
9        13.00     FALLO
10      332.00     FALLO
```

Figure 14: Compilacion, ingreso de datos y tabla de clasificacion

```
Conteo de estados:
  NORMAL: 1
  ALERTA: 2
  FALLO: 7

Lectura mas extrema:
  ID: 6 Valor: 22293.00 Estado: FALLO
vboxuser@UBUNTU:~/Documents/IE0117---Laboratorio-3-Estructuras-Y-Arreglos$
```

Figure 15: Conteo de estados y lectura mas extrema

3.3 Discusion de resultados

Con los datos de la ejecucion mostrada los resultados fueron:

ID	Valor	Estado
1	55.30	NORMAL
2	100.00	FALLO
3	322.30	FALLO
4	20.00	ALERTA
5	222.30	FALLO
6	22293.00	FALLO
7	38.00	ALERTA
8	0.00	FALLO
9	13.00	FALLO
10	332.00	FALLO

Conteo final: NORMAL: 1, ALERTA: 2, FALLO: 7. La lectura mas extrema fue el sensor 6 con valor 22293.00, cuya distancia al centro es $|22293,0 - 50,0| = 22243,0$, la mayor del conjunto.

3.4 Compilacion

```
gcc -Wall ejercicio3.c -o ej3
```

Conclusiones

- **Ejercicio 1:** El *variable shadowing* produce comportamiento incorrecto en tiempo de ejecucion sin que el compilador siempre lo senale. La validacion con `do-while` garantiza entradas consistentes.
- **Ejercicio 2:** La modularizacion en funciones separadas y el uso de la macro `SIZE` permiten que el programa escale a cualquier dimension sin modificar el cuerpo de las funciones.
- **Ejercicio 3:** El uso de `enum` y `struct` mejora la legibilidad y seguridad de tipos. La funcion auxiliar `estado_str` centraliza la conversion a texto evitando repeticion, y `fabsf()` garantiza el calculo correcto del valor absoluto en punto flotante.